# Sprint 3 Submission

## ClosetSwap

Software Engineering (CMP-N204-0)
University of Roehampton
Group Name: The Group


| | |
|---|---|
| Obaida Muqbel | A00013570 |
| Abdala Mohamed | A00012784 |
| Abdullahi Mahamud | A00017948 |
| Meshari Alsadoun | A00039102 |
| Eder Siva | A00037534 |


March 2026

SECTION

# Contents


1. Introduction
2. User Stories Implemented with Application Screenshots
3. Database Design
4. Routes Implemented in Sprint 3
5. Sprint 3 Scope Definition
6. Task Breakdown and Developer Allocation
7. GitHub Repository and Project Links
8. GitHub Contribution Metrics
9. Kanban Board
10. Meeting Records
11. Conclusion

SECTION 1

# Introduction

Sprint 3 marks the transition from planning and documentation into technical implementation. The objective of this sprint was to develop the core database-driven foundation of the ClosetSwap web application using Node.js, Express.js, MySQL, PUG templating, and Docker containers.

The focus of this sprint was to ensure that the application is dynamically connected to a database and capable of performing core CRUD operations. Rather than implementing all planned features, Sprint 3 prioritised structural stability, working routes, and proper database integration. This approach ensures that future features in Sprint 4 can be built on a reliable and scalable foundation.

The implemented application includes user registration and login with password hashing, listing creation with full CRUD functionality, dynamic browsing with category filtering, user profile pages, and a responsive PUG-based interface. All data is pulled from a MySQL database and rendered dynamically using PUG templates within Docker containers.

SECTION 2

# User Stories Implemented with Application Screenshots

## User Story 1 - User Registration [MUST HAVE]

*As a prospective user, I want to register for an account so that I can access the platform.*

**Acceptance Criteria:**

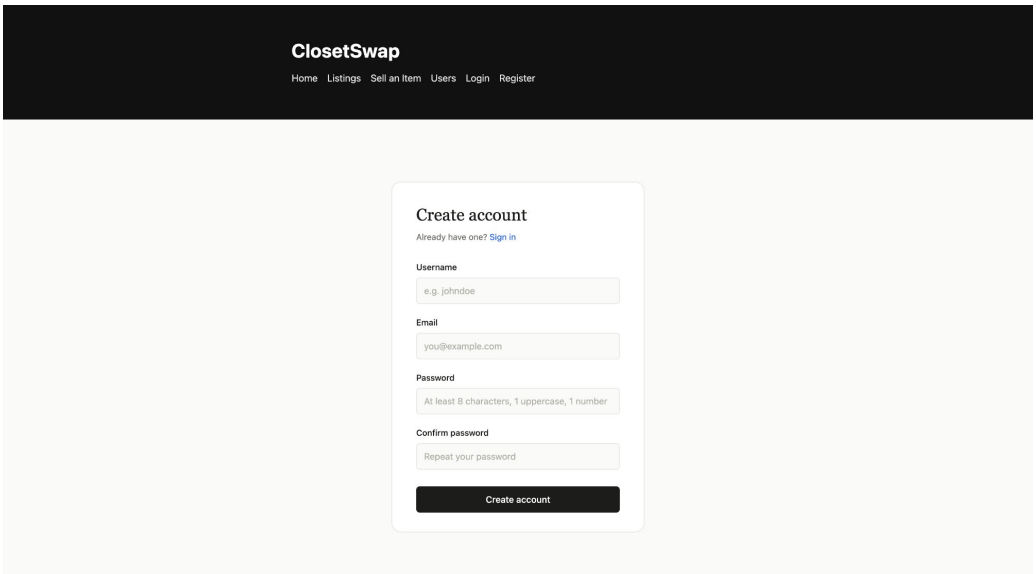
- Given a new visitor is on the registration page
- When they enter valid details (username, email, password)
- Then an account is created and stored securely in the database
- Given an email already exists, the system displays a validation error

**Implementation:**

- Registration form with server-side validation
- Password hashing using bcryptjs with 10 salt rounds
- Duplicate email and username prevention
- Auto-login after successful registration

**Application Evidence:**

*Registration page with username, email, password, and confirm password fields*




## User Story 2 - User Login [MUST HAVE]

*As a registered user, I want to log in securely so that I can access my account.*

**Acceptance Criteria:**

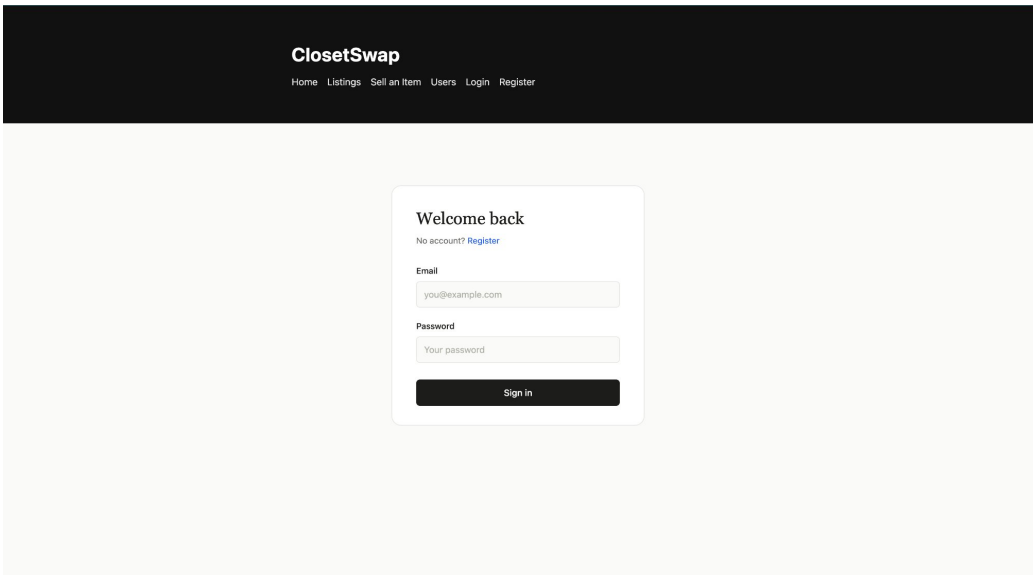
- Given valid credentials are entered, the user is authenticated
- Given invalid credentials, an error message is displayed
- Passwords are securely hashed in the database

**Implementation:**

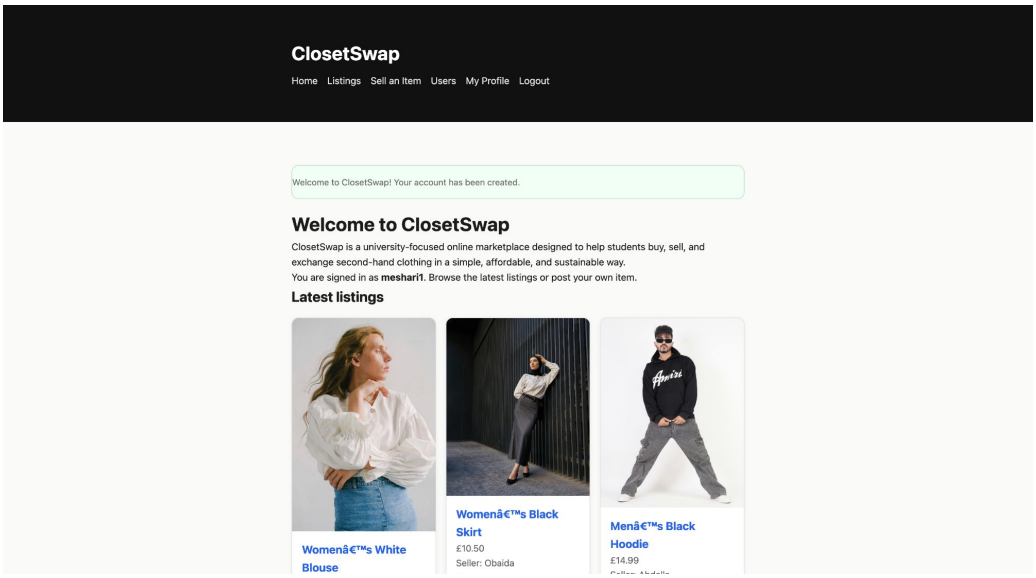
- Login form with credential verification
- bcrypt.compare() for password matching
- Session-based authentication using express-session
- Generic error messages to prevent user enumeration

### Application Evidence:

*Login page with email and password fields*




*Homepage after successful login showing welcome message and updated navbar*



## User Story 3 - Create Item Listing [MUST HAVE]

*As a seller, I want to create a listing with relevant details so that I can sell my used clothing.*

**Acceptance Criteria:**

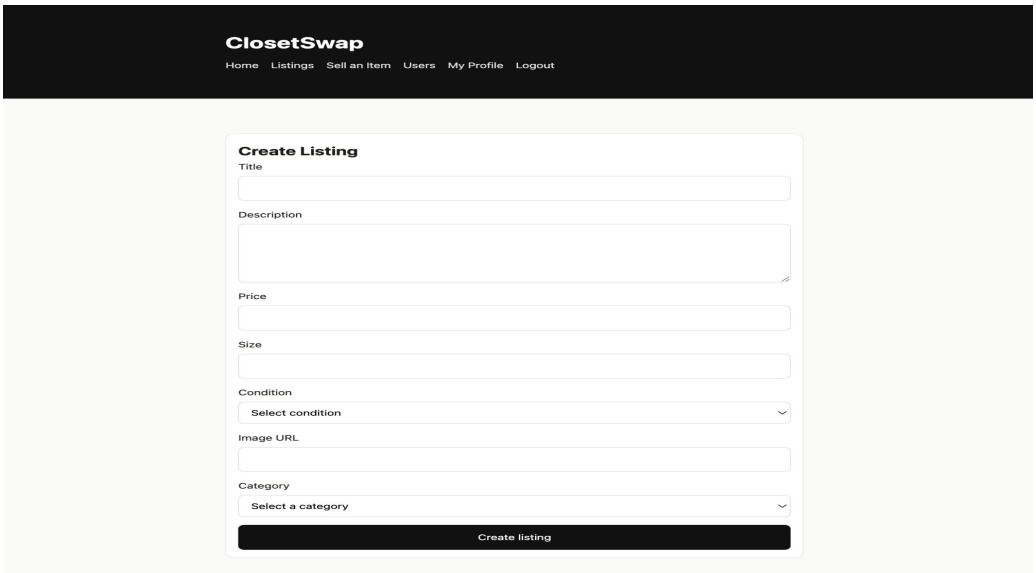
- Given a logged-in user completes the listing form
- Then the item is stored in the database and visible to other users
- All required fields must be validated before submission

**Implementation:**

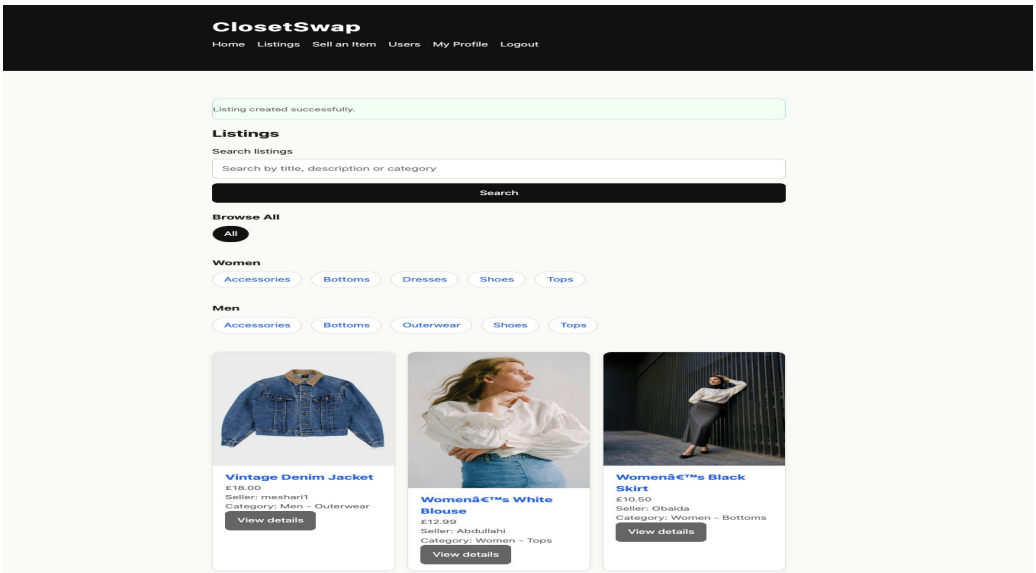
- Create listing form with field validation
- Data inserted into listings table via foreign key to logged-in user
- Authentication middleware protects the create route
- Image URL support for listing photos

**Application Evidence:**

*Create listing form with title, description, price, size, condition, image URL, and category fields*




*Listings page showing success message after creating a new listing*




## User Story 4 - Edit/Delete Listing [MUST HAVE]

*As a seller, I want to edit or remove my listings so that they remain accurate.*

**Acceptance Criteria:**

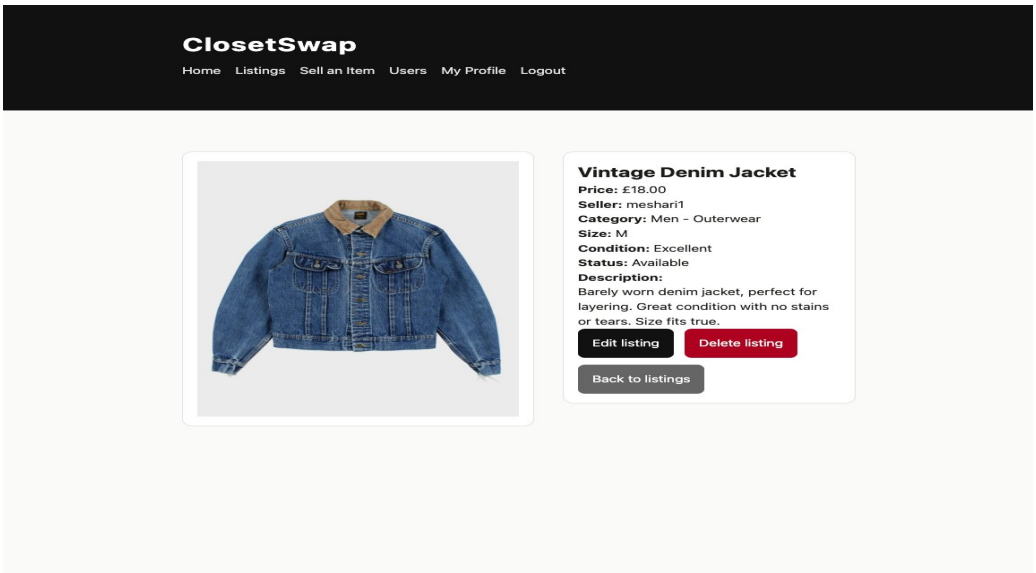
- Given the seller owns the listing, they can edit or delete it
- Users cannot modify listings they do not own
- Updated information is reflected immediately in database

**Implementation:**

- Edit form pre-populated with existing listing data
- Ownership verification before edit or delete
- POST route for deletion with redirect

**Application Evidence:**

*Listing detail page showing Edit and Delete buttons for the listing owner*




## User Story 5 - Browse Available Items [MUST HAVE]

*As a user, I want to browse available items so that I can discover sustainable fashion products.*

**Acceptance Criteria:**

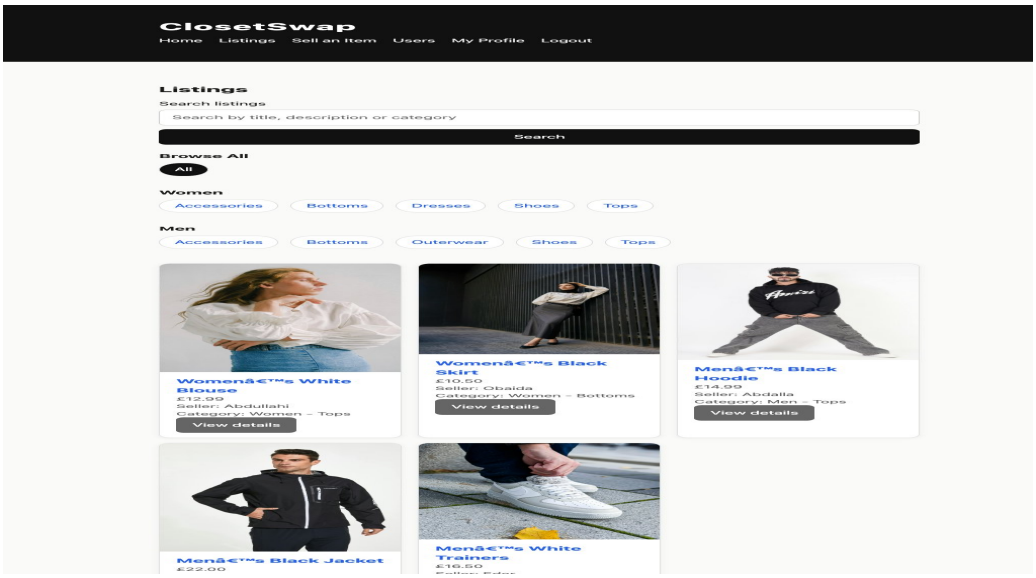
- Given a user visits the homepage, listings are displayed dynamically
- Each listing shows image, price, title, and seller

**Implementation:**

- Homepage displays latest listings from database
- Grid layout using PUG templates
- Listings page with all items and search functionality

**Application Evidence:**

*Listings page displaying all items in a grid with search bar and category filters*



## User Story 6 - Basic Category Filtering [SHOULD HAVE (Partial)]

*As a user, I want to filter items by category so that I can find relevant products.*

**Acceptance Criteria:**

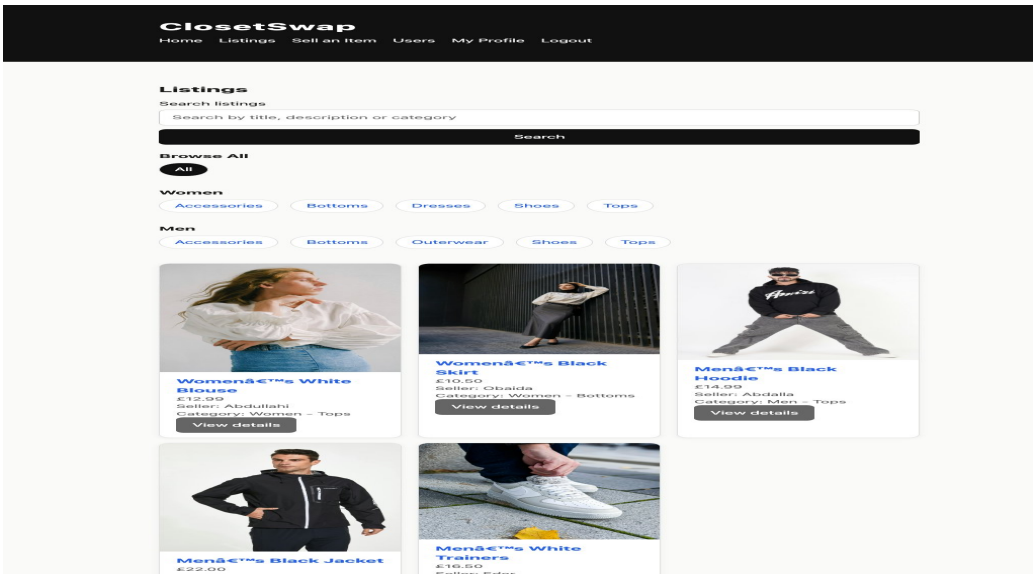
- Users can filter listings by selecting a category
- Results display only items matching the selected category

**Implementation:**

- Category route using route parameters (/categories/:id)
- Category dropdown integrated into listings page
- Advanced search deferred to Sprint 4

**Application Evidence:**

*Category filter buttons for Women and Men sections pulled from the categories table*

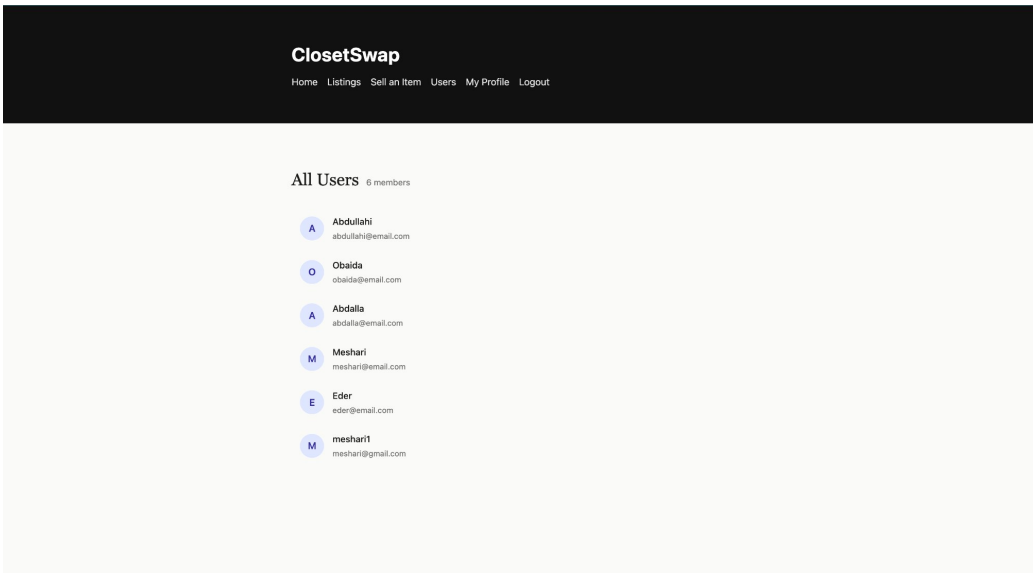



## Additional Required Pages

The following pages were also implemented as required Sprint 3 deliverables.

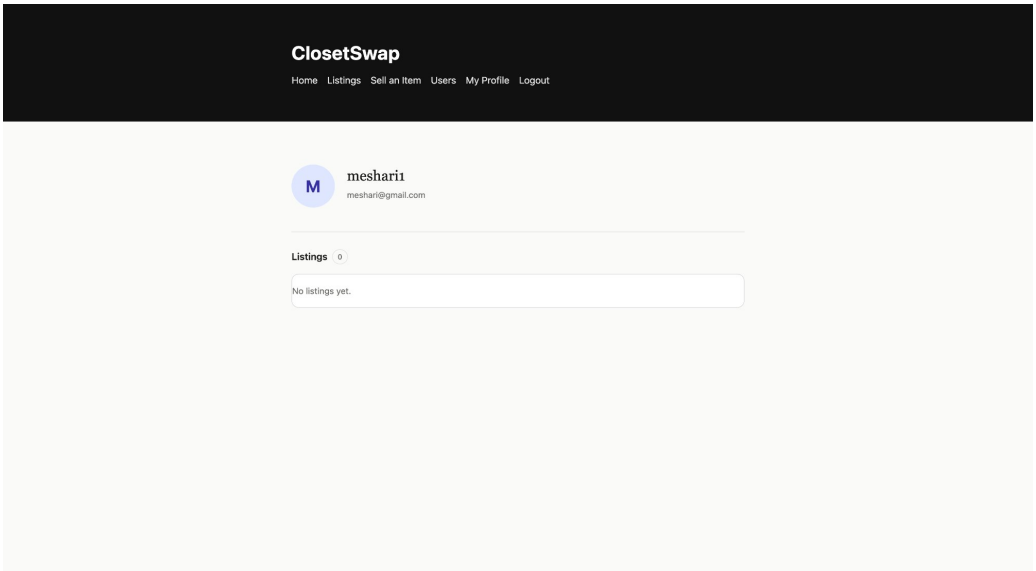
### Users List Page

*Displays all registered users pulled from the database (required Sprint 3 deliverable).*




## User Profile Page

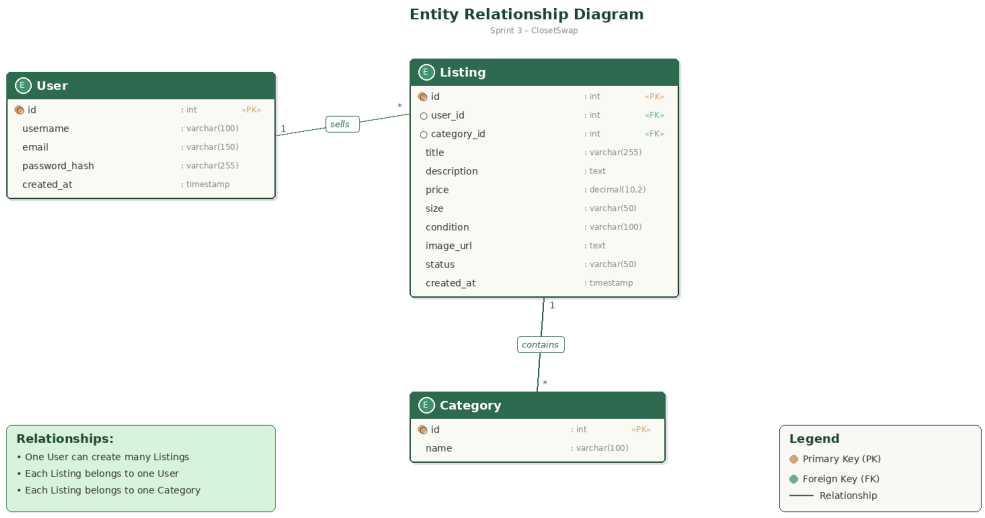
*Individual user profile showing username, email, and their listings (required Sprint 3 deliverable).*



SECTION 3

# Database Design

The database has been designed to support the implemented user stories while remaining extensible for future development. The Entity Relationship Diagram below models the three core entities implemented in Sprint 3: Users, Listings, and Categories.




Primary and foreign key relationships ensure referential integrity. The User entity links to Listings through user_id, while each Listing is associated with a Category through category_id. This relational structure supports browsing, filtering, and ownership verification. While the ERD represents the Sprint 3 scope, additional entities such as messages, orders, and feedback will be introduced in Sprint 4.

SECTION 4

# Routes Implemented in Sprint 3

The following Express routes were implemented to support the selected user stories. All routes retrieve or manipulate real data from the MySQL database and render dynamic content using PUG templates.

## 4.1 User & Authentication Routes

| Route | Method | Description |
|---|---|---|
| /register | GET | Display registration form |
| /register | POST | Handle registration with validation and bcrypt hashing |
| /login | GET | Display login form |
| /login | POST | Verify credentials and create session |
| /logout | GET | Destroy session and redirect |
| /users | GET | List all registered users |
| /users/:id | GET | View individual user profile and their listings |

## 4.2 Listing Routes

| Route | Method | Description |
|---|---|---|
| /listings | GET | Browse all listings with search functionality |
| /listings/:id | GET | View individual listing details |
| /listings/create | GET | Display create listing form (auth required) |
| /listings/create | POST | Insert new listing into database (auth required) |
| /listings/edit/:id | GET | Display edit form for owned listing (auth required) |
| /listings/edit/:id | POST | Update listing in database (auth required) |
| /listings/delete/:id | POST | Delete owned listing from database (auth required) |

## 4.3 Category Route

| Route | Method | Description |
|---|---|---|
| /categories/:id | GET | Filter listings by selected category |

## 4.4 Linking User Stories to Implementation

Each user story directly maps to both a database structure and a route, demonstrating traceability from requirement to implementation.

| User Story | Database Tables | Routes |
|---|---|---|
| Registration | users | /register |
| Login | users | /login |
| Create Listing | listings, users, categories | /listings/create |
| Edit/Delete Listing | listings | /listings/edit/:id, /listings/delete/:id |
| Browse Items | listings, users | /listings, / |
| Category Filtering | listings, categories | /categories/:id |

SECTION 5

# Sprint 3 Scope Definition

Clear scope definition was critical to ensure realistic delivery and prevent scope creep. The team deliberately focused on core functionality to establish a stable foundation before introducing additional complexity in Sprint 4.

## 5.1 Included in Sprint 3

- Fully implemented MySQL database schema (users, categories, listings)
- User registration with server-side validation and bcrypt password hashing
- User login with session-based authentication
- Authentication middleware for route protection
- Users list page and individual user profile pages
- Listing CRUD operations (create, read, edit, delete) with ownership verification
- Listings browsing page with search functionality
- Category-based filtering using route parameters
- Dynamic rendering using PUG templates
- Docker environment functioning for all team members
- Evidence of GitHub contribution from all team members

## 5.2 Not Included in Sprint 3 (Planned for Sprint 4)

- Messaging system between buyers and sellers
- Purchase and payment functionality
- Watchlist and favourites system
- Ratings and reviews
- Transaction history
- Advanced search system with multiple filters
- External API integrations
- GitHub Actions for CI/CD

This controlled scope reflects an iterative Agile approach, where value is delivered incrementally and validated at each sprint review. Establishing a working full-stack foundation in Sprint 3 reduces technical risk and allows Sprint 4 to focus on feature expansion rather than structural corrections.

SECTION 6

# Task Breakdown and Developer Allocation

Tasks were allocated to ensure clear responsibility and equal contribution across the team.

| Task | Assigned Member | Details |
|---|---|---|
| Authentication & Users | Meshari Alsadoun | Registration, login, session handling, bcrypt hashing, auth middleware, /users and /users/:id routes |
| Listings CRUD (Backend) | Abdala Mohamed | Create, read, edit, delete listings with validation, ownership checks, /categories/:id filtering |
| User Pages (Frontend) | Eder Siva | Base layout template, navigation, register, login, users list, user profile PUG pages, CSS styling |
| Listings Pages (Frontend) | Obaida Muqbel | Listings grid page, listing detail page, create and edit listing forms, category filter UI, responsive styling |
| Docker & Integration | Abdullahi Mahamud | Docker configuration, database schema and seed data, code merging, integration testing |
| Testing | All Members | Manual testing of all routes, bug fixes, cross-component verification |

Contribution is evidenced through GitHub commits and project board activity. Each member developed their part independently and the code was merged into a single working version by Obaida before the code review.

SECTION 7

# GitHub Repository and Project Links

**GitHub Repository:**

https://github.com/Abz247/Software-Engineering-coursework-1

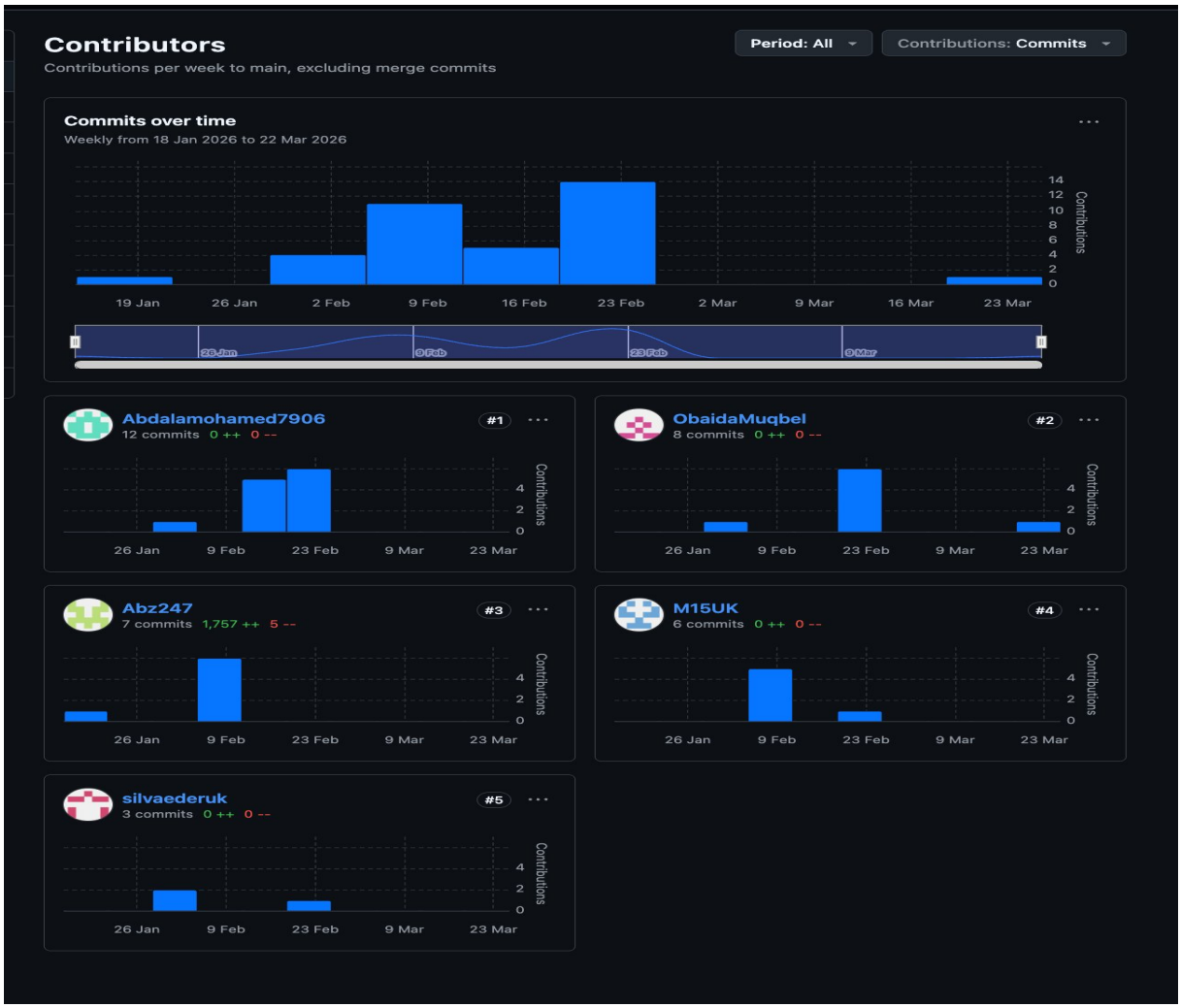
**GitHub Project Board (Kanban):**

https://github.com/users/Abz247/projects/3/views/1

SECTION 8

# GitHub Contribution Metrics

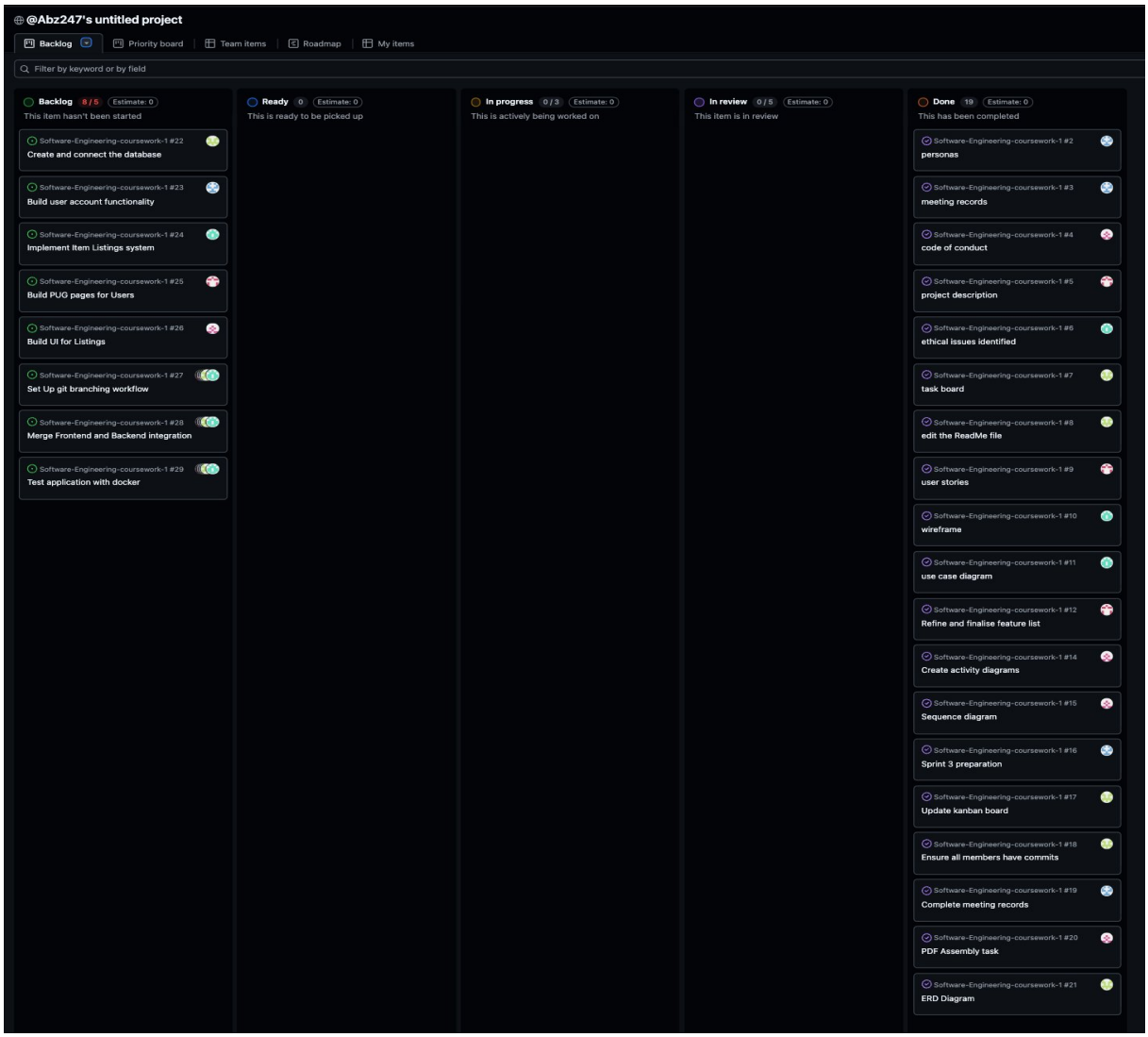
The following screenshot from GitHub Insights shows contribution activity from all team members, demonstrating collaborative participation throughout the project.



SECTION 9

# Kanban Board

The project is managed using GitHub Projects with a Kanban workflow. Tasks are categorised into Backlog, Ready, In Progress, In Review, and Done stages to ensure visibility of sprint progress.



SECTION 10

# Meeting Records

## Meeting Minutes

| | |
|---|---|
| **Date and Time** | Date: 2nd March 2026<br>Time: 13:30 |
| **Project Name** | ClosetSwap |
| **Meeting Goal** | • Begin Sprint 3 development planning<br>• Confirm task allocation for implementation<br>• Ensure Docker environment works for all members |
| **Facilitator** | Abdullahi Mahamud |
| **Note taker** | Meshari Alsadoun |
| **Attendees** | Meshari Alsadoun<br>Obaida Muqbel<br>Abdala Mohamed<br>Abdullahi Mahamud<br>Eder Siva |

| | |
|---|---|
| **Roundtable Updates** | Meshari outlined plans for the authentication system, including registration with bcrypt password hashing, login with session handling, and an authentication middleware for route protection.<br><br>Abdala discussed the listing CRUD routes and how they would connect to the database through a model layer, with validation for all form inputs and ownership checks for edit and delete.<br><br>Eder presented plans for the user-facing PUG templates including the base layout, navigation, registration form, login form, users list, and user profile pages.<br><br>Obaida outlined the listing page designs including the listings grid, listing detail page, create and edit forms, and category filtering UI, ensuring alignment with Sprint 2 wireframes.<br><br>Abdullahi confirmed Docker was working on his machine, presented the database schema with users, categories, and listings tables, and offered to handle code merging and integration. |
| **Discussion points** | The team agreed on the following task allocation: Meshari on authentication and user routes, Abdala on listing CRUD backend, Eder on user-facing PUG pages, Obaida on listing PUG pages, and Abdullahi on Docker, database schema, and code integration.<br><br>It was decided that each member would develop their part locally and share their code with Obaida, who would handle merging everything into the main codebase and resolving any conflicts.<br><br>The team agreed to focus only on core features for Sprint 3 and defer messaging, payments, and watchlists to Sprint 4. |
| **Actions** | Meshari to implement registration, login, session middleware, and user routes.<br>Abdala to implement listing create, edit, delete, and browse routes with ownership checks.<br>Eder to build PUG templates for layout, login, register, users list, and user profile pages.<br>Obaida to build PUG templates for listings grid, listing detail, create and edit listing forms.<br>Abdullahi to set up database schema, ensure Docker runs for all members, and manage code merging.<br>All members to complete their tasks and share code before the next meeting. |

# Meeting Minutes

| | |
|---|---|
| **Date and Time** | Date: 16th March 2026<br>Time: 14:00 |
| **Project Name** | ClosetSwap |
| **Meeting Goal** | • Review development progress<br>• Identify and resolve blockers<br>• Test integration between components<br>• Plan remaining work before submission |
| **Facilitator** | Abdullahi Mahamud |
| **Note taker** | Meshari Alsadoun |
| **Attendees** | Meshari Alsadoun<br>Obaida Muqbel<br>Abdala Mohamed<br>Abdullahi Mahamud<br>Eder Siva |
| **Roundtable Updates** | Meshari confirmed that registration, login, and session handling were complete and working. The authentication middleware was tested and ready for other members to use on protected routes. User list and profile pages were functional and pulling data from the database.<br><br>Abdala reported progress on listing CRUD routes. Create and browse were working. Edit and delete functionality were in progress with ownership verification logic being added.<br><br>Eder showed the completed PUG templates for layout, navigation, registration form, login form, users list page, and user profile page. Basic CSS styling was applied.<br><br>Obaida presented the listings grid page, listing detail page, and create listing form. Category filtering UI was in progress.<br><br>Abdullahi confirmed the database schema was set up in phpMyAdmin with seed data for users, categories, and sample listings. He reported that Docker was now working for all members after resolving a node_modules volume mount issue by running npm install inside the container. |

| | |
|---|---|
| **Discussion points** | The team discussed a volume mount issue where Docker was overwriting node_modules. The solution was to run npm install inside the container after docker-compose up.<br><br>Integration testing revealed that the listing routes needed to import the requireLogin middleware from Meshari's auth module to protect create, edit, and delete routes.<br><br>The group agreed that Obaida would handle the final merging of all code into one working version before the code review meeting. |
| **Actions** | Abdala to complete edit and delete listing routes with ownership checks.<br>Obaida to finalise templates for edit-listing page and category filter UI.<br>Eder to add final styling touches to user pages.<br>Abdullahi to merge all code together and resolve any integration issues.<br>All members to test the merged application locally before the review.<br>Meshari to begin preparing the Sprint 3 submission PDF. |

# Meeting Minutes

| | |
|---|---|
| **Date and Time** | Date: 27th March 2026<br>Time: 13:30 |
| **Project Name** | ClosetSwap |
| **Meeting Goal** | • Final review of merged application<br>• Confirm all Sprint 3 checklist items are complete<br>• Prepare submission PDF<br>• Ensure all members can run the application |
| **Facilitator** | Abdullahi Mahamud |
| **Note taker** | Meshari Alsadoun |
| **Attendees** | Meshari Alsadoun<br>Obaida Muqbel<br>Abdala Mohamed<br>Abdullahi Mahamud<br>Eder Siva |
| **Roundtable Updates** | Obaida confirmed the final merged version was complete. All members' code had been integrated and the application was running without errors in Docker.<br><br>Meshari verified that all authentication features were working in the merged version, including registration, login, logout, and protected routes.<br><br>Abdala confirmed listing CRUD was fully functional with create, edit, delete, and browse all working correctly. Ownership verification prevented users from modifying other users' listings.<br><br>Eder confirmed all user-facing PUG templates were rendering correctly including layout, login, register, users list, and profile pages with consistent styling.<br><br>Obaida confirmed all listing pages were displaying correctly including the listings grid, detail page, create and edit forms, and category filtering. |

| | |
|---|---|
| **Discussion points** | The team reviewed the Sprint 3 checklist to confirm all required items were implemented: users list page, user profile page, listing page, detail page, and categories.<br><br>All members confirmed they could run the application using docker-compose up --build.<br><br>The submission PDF structure was agreed: user stories, database design, task breakdown, GitHub links, contribution metrics, Kanban screenshot, and meeting records.<br><br>The team discussed Sprint 4 priorities: user login improvements, messaging, and potential GitHub Actions for CI/CD. |
| **Actions** | Meshari to compile and submit the Sprint 3 PDF.<br>Abdullahi to confirm GitHub repository is up to date with merged code.<br>Obaida to update the Kanban board to reflect completed Sprint 3 tasks.<br>All members to ensure they can run the application with docker-compose up --build.<br>All members to prepare for the Sprint 3 code review meeting. |

SECTION 11

# Conclusion

Sprint 3 establishes a fully functional, database-driven foundation for the ClosetSwap platform. Through the implementation of user registration and authentication, listing CRUD operations, category-based filtering, and dynamic PUG-based rendering, the application now supports the core user stories defined in Sprint 2.

All five team members contributed to the sprint across backend development, frontend templating, database design, and Docker integration. The application runs successfully within Docker containers, with all pages pulling real data from the MySQL database. User authentication is secured with bcrypt password hashing and session-based access control, while listing management includes ownership verification to prevent unauthorised modifications.

The project is prepared to transition into Sprint 4 with a stable architecture and clearly defined scope for feature expansion. Planned Sprint 4 work includes in-app messaging, advanced search and filtering, user ratings, and the implementation of at least one GitHub Action for CI/CD. This structured, iterative approach reflects adherence to Agile principles and supports risk-aware, incremental development.